4.1 Exercise on Functions:

Task - 1:

Create a Python program that converts between different units of measurement.

• The program should:

1. Prompt the user to choose the type of conversion (e.g., length, weight, volume).
2. Ask the user to input the value to be converted.
3. Perform the conversion and display the result.
4. Handle potential errors, such as invalid input or unsupported conversion types.

• Requirements:

1. Functions: Define at least one function to perform the conversion.
2. Error Handling: Use try-except blocks to handle invalid input (e.g., non-numeric values).
3. User Input: Prompt the user to select the conversion type and input the value.
4. Docstrings: Include a docstring in your function to describe its purpose, parameters, and return value.

• Conversion Options:

1. Length:

– Convert meters (m) to feet (ft).

– Convert feet (ft) to meters (m).

2. Weight:

– Convert kilograms (kg) to pounds (lbs).

– Convert pounds (lbs) to kilograms (kg).

3. Volume:

– Convert liters (L) to gallons (gal).

– Convert gallons (gal) to liters (L).

```
def convert_units(choice, value):
    """
    Converts a given value based on the selected conversion type.

    Parameters:
    choice (str): The type of conversion to perform.
```

```
                              Options:
                              '1' - Meters to Feet
                              '2' - Feet to Meters
                              '3' - Kilograms to Pounds
                              '4' - Pounds to Kilograms
                              '5' - Liters to Gallons
                              '6' - Gallons to Liters
    value (float): The numeric value to convert.

    Returns:
    float: The converted value.
    """

    if choice == '1':
        return value * 3.28084  # meters to feet
    elif choice == '2':
        return value / 3.28084  # feet to meters
    elif choice == '3':
        return value * 2.20462  # kg to pounds
    elif choice == '4':
        return value / 2.20462  # pounds to kg
    elif choice == '5':
        return value * 0.264172  # liters to gallons
    elif choice == '6':
        return value / 0.264172  # gallons to liters
    else:
        return None


def main():
    print("Select Conversion Type:")
    print("1. Meters (m) to Feet (ft)")
    print("2. Feet (ft) to Meters (m)")
    print("3. Kilograms (kg) to Pounds (lbs)")
    print("4. Pounds (lbs) to Kilograms (kg)")
    print("5. Liters (L) to Gallons (gal)")
    print("6. Gallons (gal) to Liters (L)")

    choice = input("Enter your choice (1-6): ")

    try:
        value = float(input("Enter the value to convert: "))

        result = convert_units(choice, value)

        if result is None:
            print("Error: Unsupported conversion type selected.")
        else:
            print(f"Converted value: {result}")

    except ValueError:
```

```
            print("Error: Please enter a valid numeric value.")

main()
```

```
Select Conversion Type:
1. Meters (m) to Feet (ft)
2. Feet (ft) to Meters (m)
3. Kilograms (kg) to Pounds (lbs)
4. Pounds (lbs) to Kilograms (kg)
5. Liters (L) to Gallons (gal)
6. Gallons (gal) to Liters (L)
Enter your choice (1-6): 1
Enter the value to convert: 34
Converted value: 111.54856
```

Task - 2:

Create a Python program that performs various mathematical operations on a list of numbers.

• The Program should:

1. Prompt the user to choose an operation (e.g., find the sum, average, maximum, or minimum of the numbers).
2. Ask the user to input a list of numbers (separated by spaces).
3. Perform the selected operation and display the result.
4. Handle potential errors, such as invalid input or empty lists.

• Requirements:

1. Functions: Define at least one function for each operation (sum, average, maximum, minimum).
2. Error Handling: Use try-except blocks to handle invalid input (e.g., non-numeric values or empty lists).
3. User Input: Prompt the user to select the operation and input the list of numbers.
4. Docstrings: Include a docstring in each function to describe its purpose, parameters, and return value.

```
def calculate_sum(numbers):
    """
    Calculates the sum of a list of numbers.

    Parameters:
    numbers (list of float): A list containing numeric values.

    Returns:
    float: The total sum of the numbers.
    """
```

```
    return sum(numbers)


def calculate_average(numbers):
    """
    Calculates the average of a list of numbers.

    Parameters:
    numbers (list of float): A list containing numeric values.

    Returns:
    float: The average of the numbers.
    """
    return sum(numbers) / len(numbers)


def find_maximum(numbers):
    """
    Finds the maximum value in a list of numbers.

    Parameters:
    numbers (list of float): A list containing numeric values.

    Returns:
    float: The largest number in the list.
    """
    return max(numbers)


def find_minimum(numbers):
    """
    Finds the minimum value in a list of numbers.

    Parameters:
    numbers (list of float): A list containing numeric values.

    Returns:
    float: The smallest number in the list.
    """
    return min(numbers)


def main():
    print("Choose an operation:")
    print("1. Find Sum")
    print("2. Find Average")
    print("3. Find Maximum")
    print("4. Find Minimum")

    choice = input("Enter your choice (1-4): ")
```

```
    try:
        user_input = input("Enter numbers separated by spaces: ")
        number_list = [float(num) for num in user_input.split()]

        if len(number_list) == 0:
            print("Error: The list cannot be empty.")
            return

        if choice == '1':
            result = calculate_sum(number_list)
            print(f"Sum: {result}")

        elif choice == '2':
            result = calculate_average(number_list)
            print(f"Average: {result}")

        elif choice == '3':
            result = find_maximum(number_list)
            print(f"Maximum: {result}")

        elif choice == '4':
            result = find_minimum(number_list)
            print(f"Minimum: {result}")

        else:
            print("Error: Invalid operation selected.")

    except ValueError:
        print("Error: Please enter only numeric values separated by spaces.")
    main()
```

```
Choose an operation:
1. Find Sum
2. Find Average
3. Find Maximum
4. Find Minimum
Enter your choice (1-4): 1
Enter numbers separated by spaces: 1 2 3 4 5 6
Sum: 21.0
```

4.2 Exercise on List Manipulation:

1. Extract Every Other Element:

Write a Python function that extracts every other element from a list, starting from the first element.

• Requirements:

– Define a function extract every other(lst) that takes a list lst as input and returns a new list containing every other element from the original list.

– Example: For the input [1, 2, 3, 4, 5, 6], the output should be [1, 3, 5].

```
def extract_every_other(lst):
    """
    Extracts every other element from a list, starting from the first element.

    Parameters:
    lst (list): The original list of elements.

    Returns:
    list: A new list containing every other element from the original list.
    """
    return lst[::2]

numbers = [1, 2, 3, 4, 5, 6]
result = extract_every_other(numbers)
print("Original List:", numbers)
print("Extracted List:", result)
```

```
Original List: [1, 2, 3, 4, 5, 6]
Extracted List: [1, 3, 5]
```

2. Slice a Sublist:

Write a Python function that returns a sublist from a given list, starting from a specified index and ending at another specified index.

• Requirements:

– Define a function get sublist(lst, start, end) that takes a list lst, a starting index start, and an ending index end as input and returns the sublist from start to end (inclusive).

– Example: For the input [1, 2, 3, 4, 5, 6] with start=2 and end=4, the output should be [3, 4, 5].

```
def get_sublist(lst, start, end):
    """
    Returns a sublist from the given list starting from index 'start'
    and ending at index 'end' (inclusive).

    Parameters:
    lst (list): The original list.
    start (int): The starting index.
    end (int): The ending index (inclusive).

    Returns:
    list: A new list containing elements from index 'start' to 'end'.
    """
```

```
    return lst[start:end + 1]

numbers = [1, 2, 3, 4, 5, 6]
result = get_sublist(numbers, 2, 4)

print("Original List:", numbers)
print("Sublist:", result)
```

```
Original List: [1, 2, 3, 4, 5, 6]
Sublist: [3, 4, 5]
```

3. Reverse a List Using Slicing:

Write a Python function that reverses a list using slicing.

• Requirements:

– Define a function reverse list(lst) that takes a list lst and returns a reversed list using slicing.

– Example: For the input [1, 2, 3, 4, 5], the output should be [5, 4, 3, 2, 1].

```
def reverse_list(lst):
    """
    Reverses a list using slicing.

    Parameters:
    lst (list): The original list to be reversed.

    Returns:
    list: A new list containing the elements in reverse order.
    """
    return lst[::-1]

numbers = [1, 2, 3, 4, 5]
result = reverse_list(numbers)

print("Original List:", numbers)
print("Reversed List:", result)
```

```
Original List: [1, 2, 3, 4, 5]
Reversed List: [5, 4, 3, 2, 1]
```

4. Remove the First and Last Elements:

Write a Python function that removes the first and last elements of a list and returns the resulting sublist.

• Requirements:

– Define a function remove first last(lst) that takes a list lst and returns a sublist without the first and last elements using slicing.

– Example: For the input [1, 2, 3, 4, 5], the output should be [2, 3, 4].

```
def remove_first_last(lst):
    """
    Removes the first and last elements of a list using slicing.

    Parameters:
    lst (list): The original list.

    Returns:
    list: A new list without the first and last elements.
    """
    return lst[1:-1]

numbers = [1, 2, 3, 4, 5]
result = remove_first_last(numbers)

print("Original List:", numbers)
print("Updated List:", result)
```

```
Original List: [1, 2, 3, 4, 5]
Updated List: [2, 3, 4]
```

5. Get the First n Elements:

Write a Python function that extracts the first n elements from a list.

• Requirements:

– Define a function get first n(lst, n) that takes a list lst and an integer n as input and returns the first n elements of the list using slicing.

– Example: For the input [1, 2, 3, 4, 5] with n=3, the output should be [1, 2, 3].

```
def get_first_n(lst, n):
    """
    Returns the first n elements from a list using slicing.

    Parameters:
    lst (list): The original list.
    n (int): The number of elements to extract from the beginning.

    Returns:
    list: A new list containing the first n elements.
    """
    return lst[:n]
```

```
numbers = [1, 2, 3, 4, 5]
result = get_first_n(numbers, 3)

print("Original List:", numbers)
print("First n Elements:", result)
```

```
Original List: [1, 2, 3, 4, 5]
First n Elements: [1, 2, 3]
```

6. Extract Elements from the End:

Write a Python function that extracts the last n elements of a list using slicing.

• Requirements:

– Define a function get last n(lst, n) that takes a list lst and an integer n as input and returns the last n elements of the list.

– Example: For the input [1, 2, 3, 4, 5] with n=2, the output should be [4, 5].

```
def get_last_n(lst, n):
    """
    Returns the last n elements from a list using slicing.

    Parameters:
    lst (list): The original list.
    n (int): The number of elements to extract from the end.

    Returns:
    list: A new list containing the last n elements.
    """
    return lst[-n:]

numbers = [1, 2, 3, 4, 5]
result = get_last_n(numbers, 2)

print("Original List:", numbers)
print("Last n Elements:", result)
```

```
Original List: [1, 2, 3, 4, 5]
Last n Elements: [4, 5]
```

7. Extract Elements in Reverse Order:

Write a Python function that extracts a list of elements in reverse order starting from the second-to-last element and skipping one element in between.

• Requirements:

– Define a function reverse skip(lst) that takes a list lst and returns a new list containing every second element starting from the second-to-last, moving backward.

– Example: For the input [1, 2, 3, 4, 5, 6], the output should be [5, 3, 1]

```
def reverse_skip(lst):
    """
    Returns every second element of the list in reverse order,
    starting from the second-to-last element.

    Parameters:
    lst (list): The original list.

    Returns:
    list: A new list containing every second element starting
          from the second-to-last element, moving backward.
    """
    return lst[-2::-2]

numbers = [1, 2, 3, 4, 5, 6]
result = reverse_skip(numbers)

print("Original List:", numbers)
print("Result:", result)
```

```
Original List: [1, 2, 3, 4, 5, 6]
Result: [5, 3, 1]
```

4.3 Exercise on Nested List:

1. Flatten a Nested List:

Write a Python function that takes a nested list and flattens it into a single list, where all the elements are in a single dimension.

• Requirements:

– Define a function flatten(lst) that takes a nested list lst and returns a flattened version of the list.

– Example: For the input [[1, 2], [3, 4], [5]], the output should be [1, 2, 3, 4, 5].

```
def flatten(lst):
    """
    Flattens a nested list into a single list (one-dimensional).

    Parameters:
    lst (list): A nested list (list of lists).

    Returns:
```

```
    list: A flattened list containing all elements in a single dimension.
    """
    flat_list = []
    for sublist in lst:
        for item in sublist:
            flat_list.append(item)
    return flat_list

nested_list = [[1, 2], [3, 4], [5]]
result = flatten(nested_list)

print("Original Nested List:", nested_list)
print("Flattened List:", result)
```

```
Original Nested List: [[1, 2], [3, 4], [5]]
Flattened List: [1, 2, 3, 4, 5]
```

2. Accessing Nested List Elements:

Write a Python function that extracts a specific element from a nested list given its indices.

• Requirements:

– Define a function access nested element(lst, indices) that takes a nested list lst and a list of indices indices, and returns the element at that position.

– Example: For the input lst = [[1, 2, 3], [4, 5, 6], [7, 8, 9]] with indices = [1, 2], the output should be 6.

```
def access_nested_element(lst, indices):
    """
    Accesses an element in a nested list using a list of indices.

    Parameters:
    lst (list): The nested list.
    indices (list of int): A list of indices specifying the path to the element

    Returns:
    any: The element located at the given indices.
    """
    element = lst
    try:
        for index in indices:
            element = element[index]
        return element
    except (IndexError, TypeError):
        raise ValueError("Invalid indices for the nested list.")

nested_list = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

```
indices = [1, 2]
result = access_nested_element(nested_list, indices)

print("Nested List:", nested_list)
print(f"Element at indices {indices}:", result)
```

```
Nested List: [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
Element at indices [1, 2]: 6
```

3. Sum of All Elements in a Nested List:

Write a Python function that calculates the sum of all the numbers in a nested list (regardless of depth).

• Requirements:

– Define a function sum nested(lst) that takes a nested list lst and returns the sum of all the elements.

– Example: For the input [[1, 2], [3, [4, 5]], 6], the output should be 21

```
def sum_nested(lst):
    """
    Calculates the sum of all numbers in a nested list, regardless of depth.

    Parameters:
    lst (list): A nested list containing numbers or other lists.

    Returns:
    int/float: The sum of all numbers in the nested list.
    """
    total = 0
    for item in lst:
        if isinstance(item, list):
            total += sum_nested(item)  #recursive
        else:
            total += item
    return total

nested_list = [[1, 2], [3, [4, 5]], 6]
result = sum_nested(nested_list)

print("Nested List:", nested_list)
print("Sum of All Elements:", result)
```

4. Remove Specific Element from a Nested List:

Write a Python function that removes all occurrences of a specific element from a nested list.

• Requirements:

– Define a function remove element(lst, elem) that removes elem from lst and returns the modified list.

– Example: For the input lst = [[1, 2], [3, 2], [4, 5]] and elem = 2, the output should be [[1], [3], [4, 5]].

```
def remove_element(lst, elem):
    """
    Removes all occurrences of a specific element from a nested list.

    Parameters:
    lst (list): A nested list (list of lists).
    elem (any): The element to remove.

    Returns:
    list: A new nested list with the element removed from all sublists.
    """
    modified_list = []
    for sublist in lst:
        if isinstance(sublist, list):
            filtered_sublist = [x for x in sublist if x != elem]
            modified_list.append(filtered_sublist)
        else:
            # If the top-level element itself matches elem, skip it
            if sublist != elem:
                modified_list.append(sublist)
    return modified_list

nested_list = [[1, 2], [3, 2], [4, 5]]
elem_to_remove = 2
result = remove_element(nested_list, elem_to_remove)

print("Original Nested List:", nested_list)
print(f"Nested List after removing {elem_to_remove}:", result)
```

```
Original Nested List: [[1, 2], [3, 2], [4, 5]]
Nested List after removing 2: [[1], [3], [4, 5]]
```

5. Find the Maximum Element in a Nested List:

Write a Python function that finds the maximum element in a nested list (regardless of depth).

• Requirements:

– Define a function find max(lst) that takes a nested list lst and returns the maximum element.

– Example: For the input [[1, 2], [3, [4, 5]], 6], the output should be 6.

```
def find_max(lst):
    """
    Finds the maximum element in a nested list, regardless of depth.

    Parameters:
    lst (list): A nested list containing numbers or other lists.

    Returns:
    int/float: The maximum number in the nested list.
    """
    max_value = float('-inf')
    for item in lst:
        if isinstance(item, list):
            max_value = max(max_value, find_max(item))  # recursive
        else:
            max_value = max(max_value, item)
    return max_value

nested_list = [[1, 2], [3, [4, 5]], 6]
result = find_max(nested_list)

print("Nested List:", nested_list)
print("Maximum Element:", result)
```

```
Nested List: [[1, 2], [3, [4, 5]], 6]
Maximum Element: 6
```

6. Count Occurrences of an Element in a Nested List:

Write a Python function that counts how many times a specific element appears in a nested list.

• Requirements:

– Define a function count occurrences(lst, elem) that counts the occurrences of elem in the nested list lst.

– Example: For the input lst = [[1, 2], [2, 3], [2, 4]] and elem = 2, the output should be 3.

```
def count_occurrences(lst, elem):
    """
    Counts how many times a specific element appears in a nested list.

    Parameters:
    lst (list): A nested list containing numbers or other lists.
    elem (any): The element to count.

    Returns:
```

```
    int: The total number of occurrences of the element.
    """
    count = 0
    for item in lst:
        if isinstance(item, list):
            count += count_occurrences(item, elem)  # recursive
        else:
            if item == elem:
                count += 1
    return count

nested_list = [[1, 2], [3, 2], [2, 5]]
elem_to_count = 2
result = count_occurrences(nested_list, elem_to_count)

print("Nested List:", nested_list)
print(f"Occurrences of {elem_to_count}:", result)
```

```
Nested List: [[1, 2], [3, 2], [2, 5]]
Occurrences of 2: 3
```

7. Flatten a List of Lists of Lists:

Write a Python function that flattens a list of lists of lists into a single list, regardless of the depth.

• Requirements:

– Define a function deep flatten(lst) that takes a deeply nested list lst and returns a single flattened list.

– Example: For the input [[[1, 2], [3, 4]], [[5, 6], [7, 8]]], the output should be [1, 2, 3, 4, 5, 6, 7, 8].

```
def deep_flatten(lst):
    """
    Flattens a deeply nested list into a single list, regardless of depth.

    Parameters:
    lst (list): A nested list of any depth.

    Returns:
    list: A flattened list containing all elements in a single dimension.
    """
    flat_list = []
    for item in lst:
        if isinstance(item, list):
            flat_list.extend(deep_flatten(item))  # recursive flatten
        else:
            flat_list.append(item)
```

```
    return flat_list

nested_list = [[[1, 2], [3, 4]], [[5, 6], [7, 8]]]
result = deep_flatten(nested_list)

print("Original Nested List:", nested_list)
print("Deep Flattened List:", result)
```

```
Original Nested List: [[[1, 2], [3, 4]], [[5, 6], [7, 8]]]
Deep Flattened List: [1, 2, 3, 4, 5, 6, 7, 8]
```

8. Nested List Average:

Write a Python function that calculates the average of all elements in a nested list.

• Requirements:

– Define a function average nested(lst) that takes a nested list lst and returns the average of all the elements.

– Example: For the input [[1, 2], [3, 4], [5, 6]], the output should be 3.5.

```
def average_nested(lst):
    """
    Calculates the average of all elements in a nested list, regardless of dept

    Parameters:
    lst (list): A nested list containing numbers or other lists.

    Returns:
    float: The average of all numeric elements.
    """
    def flatten_and_sum_count(sublist):
        total = 0
        count = 0
        for item in sublist:
            if isinstance(item, list):
                t, c = flatten_and_sum_count(item)  # recursive
                total += t
                count += c
            else:
                total += item
                count += 1
        return total, count

    total_sum, total_count = flatten_and_sum_count(lst)
    if total_count == 0:
        return 0  # avoid division by zero
    return total_sum / total_count

nested_list = [[1, 2], [3, 4], [5, 6]]
```

```
result = average_nested(nested_list)

print("Nested List:", nested_list)
print("Average of All Elements:", result)
```

```
Nested List: [[1, 2], [3, 4], [5, 6]]
Average of All Elements: 3.5
```

10 To - Do - NumPy

Please complete all the problems listed below:

10.1 Basic Vector and Matrix Operation with Numpy.

Problem - 1:

Array Creation: Complete the following Tasks:

1. Initialize an empty array with size 2X2.
2. Initialize an all one array with size 4X2.
3. Return a new array of given shape and type, filled with fill value.{Hint: np.full}
4. Return a new array of zeros with same shape and type as a given array.{Hint: np.zeros like}
5. Return a new array of ones with same shape and type as a given array.{Hint: np.ones like}
6. For an existing list new_list = [1,2,3,4] convert to an numpy array.{Hint: np.array()}

```
import numpy as np

empty_array = np.empty((2, 2))
print("1. Empty Array 2x2:\n", empty_array, "\n")

ones_array = np.ones((4, 2))
print("2. All-One Array 4x2:\n", ones_array, "\n")

shape = (3, 3)
fill_value = 7
filled_array = np.full(shape, fill_value)
print("3. Array filled with value 7:\n", filled_array, "\n")

existing_array = np.array([[1, 2, 3], [4, 5, 6]])
zeros_like_array = np.zeros_like(existing_array)
print("4. Zeros like existing array:\n", zeros_like_array, "\n")

ones_like_array = np.ones_like(existing_array)
print("5. Ones like existing array:\n", ones_like_array, "\n")

new_list = [1, 2, 3, 4]
```

```
array_from_list = np.array(new_list)
print("6. Array from list [1,2,3,4]:\n", array_from_list)
```

```
1. Empty Array 2x2:
 [[3.85388915e-315 0.00000000e+000]
 [5.73116149e-322             nan]]

2. All-One Array 4x2:
 [[1. 1.]
 [1. 1.]
 [1. 1.]
 [1. 1.]]

3. Array filled with value 7:
 [[7 7 7]
 [7 7 7]
 [7 7 7]]

4. Zeros like existing array:
 [[0 0 0]
 [0 0 0]]

5. Ones like existing array:
 [[1 1 1]
 [1 1 1]]

6. Array from list [1,2,3,4]:
 [1 2 3 4]
```

Problem - 2:

Array Manipulation: Numerical Ranges and Array indexing: Complete the following tasks:

1. Create an array with values ranging from 10 to 49. {Hint:np.arrange()}.
2. Create a 3X3 matrix with values ranging from 0 to 8. {Hint:look for np.reshape()}
3. Create a 3X3 identity matrix.{Hint:np.eye()}
4. Create a random array of size 30 and find the mean of the array. {Hint:check for np.random.random() and array.mean() function}
5. Create a 10X10 array with random values and find the minimum and maximum values.
6. Create a zero array of size 10 and replace 5th element with 1.
7. Reverse an array arr = [1,2,0,0,4,0].
8. Create a 2d array with 1 on border and 0 inside.
9. Create a 8X8 matrix and fill it with a checkerboard pattern.

```
import numpy as np

arr1 = np.arange(10, 50)
```

```
matrix3x3 = np.arange(0, 9).reshape(3, 3)

identity3x3 = np.eye(3)

random_array30 = np.random.random(30)
mean_random30 = random_array30.mean()

random_10x10 = np.random.random((10, 10))
min_10x10 = random_10x10.min()
max_10x10 = random_10x10.max()

zero_array10 = np.zeros(10)
zero_array10[4] = 1

arr = np.array([1, 2, 0, 0, 4, 0])
reversed_arr = arr[::-1]

border_array = np.ones((5, 5))
border_array[1:-1, 1:-1] = 0

checkerboard8x8 = np.zeros((8, 8), dtype=int)
checkerboard8x8[1::2, ::2] = 1
checkerboard8x8[::2, 1::2] = 1

print("Array 10 to 49:\n", arr1)
print("3x3 Matrix 0-8:\n", matrix3x3)
print("3x3 Identity Matrix:\n", identity3x3)
print("Random array of size 30:\n", random_array30)
print("Mean of random array:", mean_random30)
print("10x10 Random array:\n", random_10x10)
print("Min:", min_10x10, "Max:", max_10x10)
print("Zero array with 5th element 1:\n", zero_array10)
print("Reversed array:\n", reversed_arr)
print("2D array with border 1:\n", border_array)
print("8x8 Checkerboard pattern:\n", checkerboard8x8)
```

```
Array 10 to 49:
 [10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33
 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49]
3x3 Matrix 0-8:
 [[0 1 2]
 [3 4 5]
 [6 7 8]]
3x3 Identity Matrix:
 [[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
Random array of size 30:
 [0.24104395 0.47985546 0.48184007 0.49938303 0.56406354 0.34062553
 0.97247828 0.50173599 0.26903766 0.06652429 0.02280434 0.45533076
 0.06010304 0.28500385 0.35647378 0.86644154 0.5955551  0.01339148
 0.89377093 0.54058125 0.20539586 0.41119208 0.86511929 0.75304341
 0.58099147 0.78780155 0.8653483  0.47817063 0.50817927 0.69247751]
```

```
Mean of random array: 0.4884587752693462
10x10 Random array:
 [[0.06006441 0.22946695 0.40266229 0.07047252 0.5232314  0.29576445
  0.66722431 0.47617202 0.43332296 0.83908146]
 [0.26408987 0.22501531 0.67547897 0.74192197 0.79236099 0.81998298
  0.65244505 0.25732268 0.8146076  0.36902574]
 [0.03231608 0.72407684 0.66788612 0.190502   0.44486634 0.7516487
  0.80219719 0.7499634  0.62044213 0.37206321]
 [0.84818998 0.2339503  0.15723056 0.82495035 0.28646841 0.84831578
  0.10813712 0.44795758 0.5128933  0.52974437]
 [0.14181638 0.45088578 0.85644266 0.06897476 0.44496755 0.68745311
  0.79488638 0.65131486 0.08563193 0.29423991]
 [0.3090304  0.12128621 0.30181462 0.26721611 0.66522914 0.71978129
  0.74352034 0.25570805 0.99339031 0.25538262]
 [0.48525342 0.9151436  0.5836397  0.96250458 0.89570038 0.09346975
  0.45318646 0.9060579  0.72777061 0.05921398]
 [0.54614546 0.11902342 0.1724024  0.55155274 0.87628743 0.93022014
  0.20997649 0.76766507 0.45664222 0.55038637]
 [0.54373608 0.21330371 0.90197829 0.04915508 0.43427485 0.13708486
  0.35868801 0.49571655 0.873127   0.82812668]
 [0.32481234 0.46908203 0.66782359 0.3172444  0.97526068 0.12694219
  0.7624933  0.06286177 0.33437774 0.31492601]]
Min: 0.032316075685587586 Max: 0.9933903088461922
Zero array with 5th element 1:
 [0. 0. 0. 0. 1. 0. 0. 0. 0. 0.]
Reversed array:
 [0 4 0 0 2 1]
2D array with border 1:
 [[1. 1. 1. 1. 1.]
 [1. 0. 0. 0. 1.]
 [1. 0. 0. 0. 1.]
 [1. 0. 0. 0. 1.]
 [1. 1. 1. 1. 1.]]
8x8 Checkerboard pattern:
 [[0 1 0 1 0 1 0 1]
 [1 0 1 0 1 0 1 0]
 [0 1 0 1 0 1 0 1]
 [1 0 1 0 1 0 1 0]
 [0 1 0 1 0 1 0 1]
 [1 0 1 0 1 0 1 0]
 [0 1 0 1 0 1 0 1]
```

Problem - 3:

Array Operations:

For the following arrays:

x = np.array([[1,2],[3,5]]) and y = np.array([[5,6],[7,8]]); v = np.array([9,10]) and w = np.array([11,12]);

Complete all the task using numpy:

1. Add the two array.
2. Subtract the two array.

3. Multiply the array with any integers of your choice.
4. Find the square of each element of the array.
5. Find the dot product between: v(and)w ; x(and)v ; x(and)y.
6. Concatenate x(and)y along row and Concatenate v(and)w along column. {Hint:try np.concatenate() or np.vstack() functions.
7. Concatenate x(and)v; if you get an error, observe and explain why did you get the error?

```
import numpy as np

x = np.array([[1, 2], [3, 5]])
y = np.array([[5, 6], [7, 8]])
v = np.array([9, 10])
w = np.array([11, 12])

add_xy = x + y
sub_xy = x - y
mul_x3 = x * 3
square_x = x ** 2
dot_vw = np.dot(v, w)
dot_xv = np.dot(x, v)
dot_xy = np.dot(x, y)
concat_row_xy = np.concatenate((x, y), axis=0)
concat_col_vw = np.concatenate((v.reshape(-1, 1), w.reshape(-1, 1)), axis=1)

try:
    concat_xv = np.concatenate((x, v), axis=0)
except ValueError as e:
    concat_xv_error = e

print("x + y:\n", add_xy)
print("x - y:\n", sub_xy)
print("x * 3:\n", mul_x3)
print("Square of x:\n", square_x)
print("Dot product v·w:", dot_vw)
print("Dot product x·v:\n", dot_xv)
print("Dot product x·y:\n", dot_xy)
print("Concatenate x and y along row:\n", concat_row_xy)
print("Concatenate v and w along column:\n", concat_col_vw)
print("Concatenate x and v error:\n", concat_xv_error)
```

```
x + y:
 [[ 6  8]
 [10 13]]
x - y:
 [[-4 -4]
 [-4 -3]]
x * 3:
 [[ 3  6]
```

```
 [ 9 15]]
Square of x:
 [[ 1  4]
 [ 9 25]]
Dot product v·w: 219
Dot product x·v:
 [29 77]
Dot product x·y:
 [[19 22]
 [50 58]]
Concatenate x and y along row:
 [[1 2]
 [3 5]
 [5 6]
 [7 8]]
Concatenate v and w along column:
 [[ 9 11]
 [10 12]]
Concatenate x and v error:
 all the input arrays must have same number of dimensions, but the array at inde
```

Problem - 4:

Matrix Operations:

• For the following arrays:

A = np.array([[3,4],[7,8]]) and B = np.array([[5,3],[2,1]]);

Prove following with Numpy:

1. Prove A.A−1 = I.
2. Prove AB /= BA.
3. Prove (AB) T = BTAT .

• Solve the following system of Linear equation using Inverse Methods.

2x − 3y + z = −1 x − y + 2z = −3 3x + y − z = 9

{Hint: First use Numpy array to represent the equation in Matrix form. Then Solve for: AX = B}

• Now: solve the above equation using np.linalg.inv function.{Explore more about ”linalg” function of Numpy}

```
import numpy as np

A = np.array([[3, 4], [7, 8]])
B = np.array([[5, 3], [2, 1]])

# 1. Prove A · A^-1 = I
A_inv = np.linalg.inv(A)
```

```
identity = np.dot(A, A_inv)

# 2. Prove AB ≠ BA
AB = np.dot(A, B)
BA = np.dot(B, A)
are_equal = np.array_equal(AB, BA)

# 3. Prove (AB)^T = B^T A^T
AB_transpose = (np.dot(A, B)).T
B_transpose_A_transpose = np.dot(B.T, A.T)
transpose_equal = np.array_equal(AB_transpose, B_transpose_A_transpose)

# Solve system of equations using inverse method
coeff_matrix = np.array([[2, -3, 1],
                         [1, -1, 2],
                         [3, 1, -1]])
const_vector = np.array([-1, -3, 9])

solution = np.dot(np.linalg.inv(coeff_matrix), const_vector)

print("A · A^-1:\n", identity)
print("AB:\n", AB)
print("BA:\n", BA)
print("AB equals BA?:", are_equal)
print("(AB)^T:\n", AB_transpose)
print("B^T A^T:\n", B_transpose_A_transpose)
print("(AB)^T equals B^T A^T?:", transpose_equal)
print("Solution to the system of equations [x, y, z]:", solution)
```

```
A · A^-1:
 [[1.00000000e+00 0.00000000e+00]
 [1.77635684e-15 1.00000000e+00]]
AB:
 [[23 13]
 [51 29]]
BA:
 [[36 44]
 [13 16]]
AB equals BA?: False
(AB)^T:
 [[23 51]
 [13 29]]
B^T A^T:
 [[23 51]
 [13 29]]
(AB)^T equals B^T A^T?: True
Solution to the system of equations [x, y, z]: [ 2.  1. -2.]
```

10.2 Experiment: How Fast is Numpy?

In this exercise, you will compare the performance and implementation of operations using plain Python lists (arrays) and NumPy arrays. Follow the instructions:

1. Element-wise Addition: • Using Python Lists, perform element-wise addition of two lists of size 1, 000, 000. Measure and Print the time taken for this operation. • Using Numpy Arrays, Repeat the calculation and measure and print the time taken for this operation.
2. Element-wise Multiplication • Using Python Lists, perform element-wise multiplication of two lists of size 1, 000, 000. Measure and Print the time taken for this operation. • Using Numpy Arrays, Repeat the calculation and measure and print the time taken for this operation.
3. Dot Product • Using Python Lists, compute the dot product of two lists of size 1, 000, 000. Measure and Print the time taken for this operation. • Using Numpy Arrays, Repeat the calculation and measure and print the time taken for this operation.
4. Matrix Multiplication • Using Python lists, perform matrix multiplication of two matrices of size 1000x1000. Measure and print the time taken for this operation. • Using NumPy arrays, perform matrix multiplication of two matrices of size 1000x1000. Measure and print the time taken for this operation.

```
import numpy as np
import time

size = 1_000_000

# Python Lists
list1 = list(range(size))
list2 = list(range(size))

# Element-wise addition (Python Lists)
start = time.time()
list_add = [a + b for a, b in zip(list1, list2)]
end = time.time()
print("Python List Addition Time:", end - start, "seconds")

# Element-wise addition (NumPy)
arr1 = np.arange(size)
arr2 = np.arange(size)
start = time.time()
np_add = arr1 + arr2
end = time.time()
print("NumPy Array Addition Time:", end - start, "seconds")

# Element-wise multiplication (Python Lists)
start = time.time()
list_mul = [a * b for a, b in zip(list1, list2)]
end = time.time()
print("Python List Multiplication Time:", end - start, "seconds")
```

```
# Element-wise multiplication (NumPy)
start = time.time()
np_mul = arr1 * arr2
end = time.time()
print("NumPy Array Multiplication Time:", end - start, "seconds")

# 3Dot product (Python Lists)
start = time.time()
dot_list = sum(a * b for a, b in zip(list1, list2))
end = time.time()
print("Python List Dot Product Time:", end - start, "seconds")

# Dot product (NumPy)
start = time.time()
dot_np = np.dot(arr1, arr2)
end = time.time()
print("NumPy Dot Product Time:", end - start, "seconds")

# Matrix multiplication
matrix_size = 1000

# Python Lists
matrix1 = [[i + j for j in range(matrix_size)] for i in range(matrix_size)]
matrix2 = [[i + j for j in range(matrix_size)] for i in range(matrix_size)]

start = time.time()
result_list = [[sum(matrix1[i][k] * matrix2[k][j] for k in range(matrix_size))
                for j in range(matrix_size)] for i in range(matrix_size)]
end = time.time()
print("Python List Matrix Multiplication Time:", end - start, "seconds")

# NumPy Arrays
matrix_np1 = np.arange(matrix_size*matrix_size).reshape(matrix_size, matrix_siz
matrix_np2 = np.arange(matrix_size*matrix_size).reshape(matrix_size, matrix_siz

start = time.time()
result_np = np.dot(matrix_np1, matrix_np2)
end = time.time()
print("NumPy Matrix Multiplication Time:", end - start, "seconds")
```

```
Python List Addition Time: 0.1254429817199707 seconds
NumPy Array Addition Time: 0.005313873291015625 seconds
Python List Multiplication Time: 0.13487696647644043 seconds
NumPy Array Multiplication Time: 0.0052912235260009766 seconds
Python List Dot Product Time: 0.17464756965637207 seconds
NumPy Dot Product Time: 0.0021216869354248047 seconds
Python List Matrix Multiplication Time: 209.15411233901978 seconds
NumPy Matrix Multiplication Time: 1.4388806819915771 seconds
```